Phase 1: Data Collection and Tokenizer Construction

Monolingual Transformer Language Models for Gujarati and Nepali

Hardik Kothari

Abstract

This phase builds two independent monolingual corpora and two independent tokenizers: Gujarati as the higher-resource language (Model H) and Nepali from the allowed lower-resource list (Model L). The Gujarati corpus contains **573.7 million tokens**, of which **31.44%** were collected manually; the Nepali corpus contains **542.0 million tokens** with **21.58%** manual. Both exceed the 500M token target and both clear the 20% manual-collection requirement. Each language has its own SentencePiece Unigram tokenizer with a vocabulary of 16,000 pieces, selected on held-out text against a parameter budget rather than on compression alone. The two models share no documents, no tokenizer, no vocabulary and no training text.

Corpus download (raw and preprocessed, both languages, plain `.jsonl`):
iiithydstudents-my.sharepoint.com/…/hardik_k_students

Tokenizer models and vocabularies are committed to the repository directly.

1 Language selection

Model H — Gujarati. Gujarati occupies an exclusive Unicode block (U+0A80–U+0AFF) that no other language uses. Language identification is therefore an exact character test rather than a probabilistic one, which matters because the brief does not permit pretrained models. Public Gujarati text is also plentiful, so the token target is reachable while still leaving room for a large manually collected share.

Model L — Nepali. Nepali is on the permitted lower-resource list and has a substantial public web presence. It is harder than Gujarati in one specific respect that shaped much of the language-specific code: Nepali shares the Devanagari script with Hindi, Marathi and Sanskrit and has no exclusive characters, so identification cannot be based on script (Section 5).

2 Corpus collection

Each corpus combines one public dataset (AI4Bharat Sangraha, streamed for the relevant language subset only) with sources collected directly. Collection is implemented as four interchangeable adapters under `data/collection/adapters/`, selected per source by a `method:` field in the source registry: a site's own JSON article endpoint where one exists, sitemap-driven crawling and link discovery, archive.org full-text retrieval, and text-layer extraction from locally downloaded PDFs.

Which adapter a source needs is decided by the site, not by preference. Some publishers expose a documented JSON endpoint that returns complete article bodies; most Nepali news sites run custom content-management systems with no equivalent, so 20 of the 23 Nepali sources had to be fetched a page at a time and parsed from HTML. That difference is why Nepali needed nearly twice as many sources to produce less text.

	Gujarati	Nepali
collected directly (13 and 23 sources)	**180,350,586**	**116,938,372**
public dataset (AI4Bharat Sangraha)	393,316,557	425,022,332
total	**573,667,143**	**541,960,704**

Table 1: Corpus tokens by provenance, counted across all three splits.

Every source is treated the same way regardless of adapter: the source is chosen and checked for language and terms of use, `robots.txt` is re-verified on every run rather than recorded once, markup and site boilerplate are stripped, and the text is filtered on script ratio before it is written. Politeness is implemented as a per-host concurrency ceiling rather than a delay between requests.

2.1 Source composition

Source	Register	Documents	Tokens
Gujarati — 13 manual sources			
`ia_books_fulltext`	books, all eras	868	48,123,533
`abtakmedia`	regional news	65,810	38,108,858
`gujarativishwakosh`	encyclopedia	20,623	22,568,398
`westerntimesnews`	city news, lifestyle	46,022	22,143,962
`opinionmagazine`	opinion, essays	11,149	18,015,441
`gujarattoday`	general news	44,868	16,365,808
7 further sources (literature, poetry, diaspora, PDFs)		18,239	15,024,586
Nepali — 23 manual sources			
`onlinekhabar`	news	59,094	29,989,965
`sahityasangraha_html`	literature	12,877	19,794,023
`nepalpress_html`	news	39,934	14,018,135
`reportersnepal_html`	news	39,336	13,516,434
`khabarhub_html`	news	39,842	13,183,948
`clickmandu_html`	business	18,185	7,074,374
17 further sources (culture, finance, health, legal, …)		51,289	19,361,493

Table 2: The largest manual sources in each corpus, over all splits.

The smaller sitemap sources cost roughly 80 times more time per character than the REST sources and contribute a few percent each. They are retained for register diversity: a corpus assembled only from news sites represents a narrow set of house styles, and a tokenizer fitted to it handles academic or literary prose poorly. The effect is measurable in how differently each register compresses — Gujarati book text runs at 2.97 characters per token against 4.31 for news — so the books and the encyclopedia contribute vocabulary that news prose does not contain.

The Nepali source list is broad but thin outside news. It spans 11 non-news categories, yet health, education, agriculture and legal together account for under 1.5% of the corpus. The legal source illustrates the limit: `court_pdf` yielded 22 usable documents, because roughly 88% of the Supreme Court PDFs are scanned images carrying no text layer and no Nepali OCR model was available.

3 Cleaning and normalisation

Cleaning is deliberately split across two stages. Collection applies a loose quality gate and preprocessing applies the strict one, because rejecting a page at collection time is irreversible — it would have to be re-crawled — while rejecting it during preprocessing is a configuration change and a local re-run.

Normalisation. NFC is used rather than NFKC: NFC composes nukta letters into single code points so the tokenizer sees one spelling, whereas NFKC would additionally collapse distinctions that carry meaning. ZWJ (U+200D) is removed but **ZWNJ (U+200C) is preserved**, because ZWNJ can block a conjunct and therefore changes the word. URLs and email addresses are stripped, danda spacing is regularised for Nepali, and typographic quotes and dashes are folded to ASCII.

Two rules follow an explicit ruling from the teaching staff. Every Latin-script word is removed, including proper nouns such as `Tata` and `IRCTC`, so that each corpus is strictly monolingual. Digits are retained but written in the language's own script: `2024` becomes `૨૦૨૪` in Gujarati and `२०२४` in Nepali. The digit rule reverses an earlier decision in this pipeline, which had folded native digits to ASCII on the grounds that they would otherwise occupy vocabulary slots.

Latin removal operates at the level of individual words rather than whole documents, and the distinction is consequential. Measured over 5,057 raw documents, **41.9% contain at least one Latin word, but Latin accounts for only 0.99% of characters**. Discarding those documents would remove 41.9% of the corpus in order to eliminate 1% of the text, placing both languages well below the token target. The limitation of the word-level rule is that a sentence containing an inflected loanword is left with a stranded case marker; documents degraded far enough are removed downstream by the minimum-word and repetition filters.

Document filters. A document is rejected if it falls below the script threshold, exceeds the Latin or digit ratios, is too short, repeats lines or n-grams excessively, or lacks the common function words that ordinary prose contains. The last check is what removes navigation menus and name lists, which otherwise pass every statistical test.

Boilerplate removal operates in three layers, because no single layer is sufficient. `trafilatura` reads page structure and discards navigation and comment sections. A language-specific rule strips "also read" link lists, which content-management themes inject *inside* the article body where `trafilatura` correctly preserves them. Finally, a corpus-wide pass blocks any line appearing in 50 or more documents. Only the third layer can identify boilerplate it was never told about, and it can only run during preprocessing, since "appears in 50 documents" is not observable from within a single document.

4 Deduplication

Deduplication runs **after** cleaning and **before** splitting. The ordering is the important part: if a near-duplicate of a test document survives into the training split, every Phase 2 perplexity figure is inflated and meaningless.

Two passes are applied. Exact duplicates are removed first by SHA1 of the whitespace-collapsed, case-folded text, which is cheap and removes the bulk. Near-duplicates are then removed using MinHash with locality-sensitive hashing over 5-word shingles at a Jaccard threshold of 0.8 and 128 permutations, which catches syndicated news republished under a changed headline. Comparing every document against every other is infeasible at this scale, so MinHash reduces each document to a short signature and LSH compares only those documents whose signatures are already close.

Manual documents are streamed first, so that when a manual and a public document collide the manual copy is the one retained and the manual share is not eroded by deduplication.

	Gujarati		Nepali	
Stage	Documents	% of raw	Documents	% of raw
collected (raw)	864,122	100.0%	1,378,885	100.0%
after cleaning	843,660	97.6%	1,255,276	91.0%
after deduplication	830,410	96.1%	1,239,652	89.9%
exact duplicates removed	3,365		5,045	
near-duplicates removed	9,885		10,579	

Table 3: Documents surviving cleaning and deduplication. The gap between the two languages is almost entirely the Nepali language filter (Section 5).

Deduplication removes only 1.6% and 1.2% respectively, far below the 20–50% commonly reported. That figure was checked rather than assumed: re-running deduplication on a 60,000-document sample at a stricter 256 permutations left 0.017% surviving duplicates, so the low rate is genuine. Sangraha is distributed already deduplicated, collection through APIs and sitemaps returns each article exactly once, and the commonly quoted figures describe raw Common Crawl, which is not the input here.

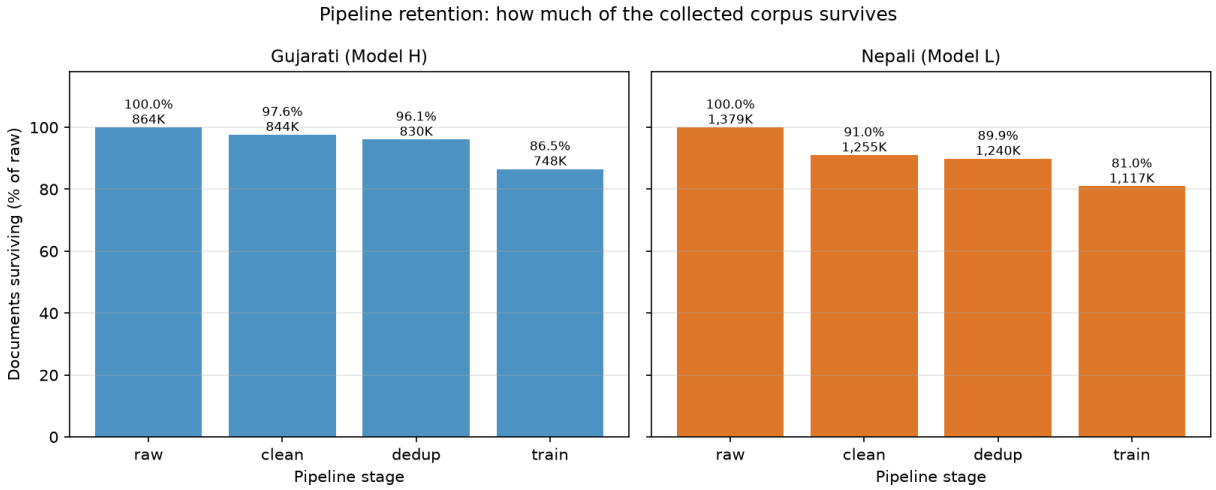


Figure 1: Documents surviving each pipeline stage. The final step is the 90/5/5 split rather than a filter.

5 Language identification for Nepali

Gujarati occupies its own Unicode block, so the test "at least 85% of letters lie in U+0A80–U+0AFF" is complete and exact. Nepali has no exclusive characters at all, and a Hindi page satisfies any Devanagari check perfectly. Because pretrained models are not permitted, identification had to be lexical:

1. the text must be at least 80% Devanagari, which excludes other scripts;
2. it must contain Nepali function words — छ, छन्, हो, भएको, गरेको — at a minimum rate per 1000 characters;

3. Nepali markers must outnumber Hindi markers (है, हैं, था) by at least 1.5 to 1.

Measured on real pages, Nepali news scores 11–14 Nepali markers per 1000 characters and effectively zero Hindi markers, while Hindi pages invert that ratio exactly. This filter is the single most expensive component of the Nepali pipeline: it rejected **102,137 documents** that are genuinely Devanagari and still not Nepali. The equivalent Gujarati categories total 11,788 — a ninth as many — and are caught by the script test at no additional cost.

6 Dataset statistics

6.1 Splits

Splits are assigned per document by hashing the document identifier with a fixed seed, `md5(seed:doc_id)`. Assignment is at document level and never at sentence level, so no article is divided between training and test, and it is deterministic, so the pipeline reproduces exactly and Phase 2 and Phase 3 remain comparable across re-runs. The ratio is 90/5/5; the character caps in the configuration are set above what 5% amounts to, so the fraction is what binds.

	Gujarati (Model H)			Nepali (Model L)		
Split	Documents	Tokens	Manual	Documents	Tokens	Manual
train	747,765	517,108,008	31.51%	1,116,564	488,046,826	21.59%
validation	41,171	28,416,795	31.64%	61,604	26,814,777	21.18%
test	41,474	28,142,340	29.95%	61,484	27,099,101	21.68%
corpus	**830,410**	**573,667,143**	**31.44%**	**1,239,652**	**541,960,704**	**21.58%**

Table 4: Corpus size and composition by split. Token counts are exact, produced by encoding every document with that language's final tokenizer.

The manual share agrees to within 1.6 percentage points across the three splits in each language, which is the expected behaviour of a hash-based assignment: provenance is uncorrelated with the hash, so it carries through the split unchanged.

6.2 Corpus summary

Whitespace words and tokens should not be confused. Gujarati's 370.0M words become 573.7M tokens and Nepali's 380.6M become 542.0M, because subword tokenization divides inflected forms into several pieces. The target is defined in tokens, so the lower word counts are expected rather than a shortfall.

On the training split. The brief phrases the target as "approximately 500M training tokens". Read strictly as the training split alone, Gujarati reaches 517.1M and Nepali reaches 488.0M, which is 11.95M short. The Nepali corpus itself clears the target comfortably; the difference is that 10% of it is held out for validation and test. That ratio was chosen deliberately so that Phase 2 has a validation set spanning all 23 sources — 61,604 documents rather than the roughly 3,900 a 0.35% split would give. Section 1.2 of the brief permits the lower-resource language to report the maximum feasible amount with a justification, and a 96/2/2 split would return the training split to approximately 520M if the stricter reading is required.

	Gujarati (Model H)	Nepali (Model L)
Documents collected	864,122	1,378,885
Documents after cleaning and deduplication	830,410	1,239,652
Characters	2,165,416,159	2,528,493,333
Whitespace words	369,994,209	380,622,400
Total tokens	**573,667,143**	**541,960,704**
Against the 500M target	114.7%	108.4%
Manual tokens	180,350,586	116,938,372
Public tokens	393,316,557	425,022,332
Manual share	**31.44%**	**21.58%**
Manual sources	13	23
Collection methods	4	3
Vocabulary size	16,000	16,000

Table 5: Both corpora exceed the 500M token target and both exceed the 20% manual-collection requirement.

6.3 Provenance

Every document carries `source_type`, `collection_method` and `manual_or_public` from the moment it is collected, and those fields survive cleaning, deduplication and splitting. The manual share is therefore measured rather than estimated.

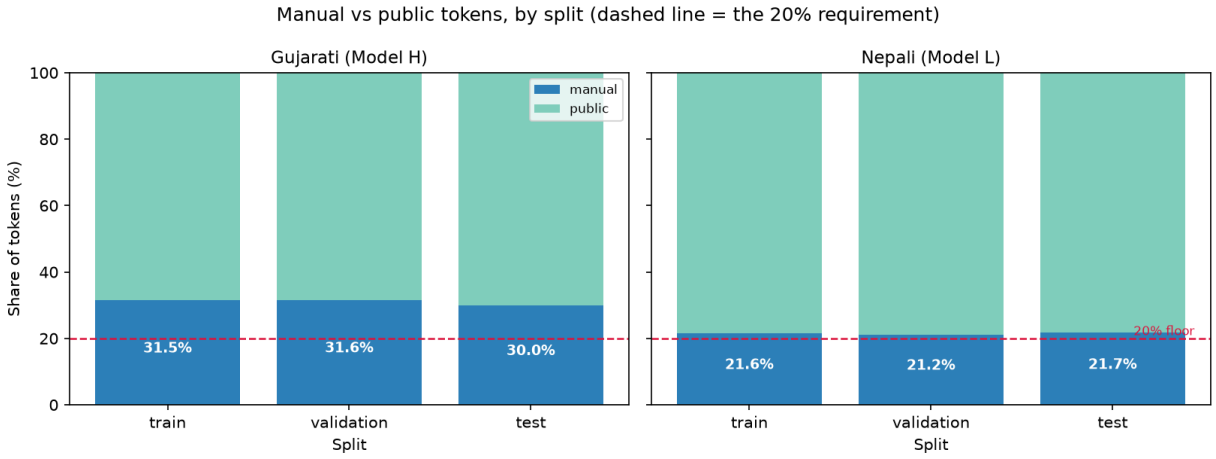


Figure 2: Manual share of each split against the 20% requirement. All six splits clear it.

Two labels are deliberately weaker than they could be. `ia_books_pdf` and `court_pdf` are recorded as `manual_pdf` rather than `manual_ocr`, because they read a text layer already embedded in the PDF and no optical recognition is performed. `ia_books_fulltext` is recorded as `manual_web` rather than `manual_ocr`, because archive.org performed that OCR on its own servers and this project downloaded and cleaned the result. All three count toward the 20% requirement under either label, so the weaker description costs nothing and keeps the statistics accurate.

One point should be stated explicitly. The teaching staff indicated that data arriving in a structured format might count toward the public 80% rather than the manual 20%. On the strictest possible reading of that, the manual share would fall to 13.34% for Gujarati and 15.53% for

Nepali. The counter-argument is that this project selected each source, verified its language and terms of use, fetched and paginated it, stripped markup, removed site boilerplate, filtered on script ratio and deduplicated the result — none of which is required when using a packaged corpus release, which is what the `public_hf` label denotes.

7 Tokenizer construction

Each language has its own SentencePiece tokenizer, trained from scratch on that language's training split alone. No pretrained tokenizer is used, and no training text, vocabulary or piece inventory is shared between the two.

7.1 Choice of algorithm

Unigram was selected over BPE for both languages, for three reasons.

Segmentation at morpheme boundaries. Gujarati and Nepali mark case and number with chains of suffixes. Unigram scores a complete segmentation under a probabilistic model over the sentence, so it tends to cut where the morphology cuts. BPE merges the most frequent pair greedily, producing frequent character sequences that need not align with meaning.

Stability of the stem across inflections. This matters directly for Phase 2. If ઘરમાં and ઘરોમાં are segmented such that the stem differs between them, the model must learn the same word twice and its evidence is divided across two embedding rows. Section 7.4 shows the stem surviving intact across five inflected forms.

A practical constraint. The `required_chars` option is incompatible with the SentencePiece BPE trainer, which fails when a required character has zero frequency in the training text. Pinning every code point of the script into the vocabulary — so that the tokenizer can never fall back to bytes on its own letters — is therefore available only under Unigram.

Other settings follow from the pipeline. Normalisation is set to `identity`, because preprocessing has already normalised the text and a second, different normalisation would leave the tokenizer mismatched to the corpus. Byte fallback is enabled, so nothing is unrepresentable. Splitting by Unicode script prevents pieces straddling two writing systems, and splitting digits prevents specific years being memorised as single pieces.

7.2 Choice of vocabulary size

Five candidate sizes were trained per language and evaluated on the **complete** validation split of that language — 41,171 Gujarati documents totalling 107.3M characters, and 61,604 Nepali documents totalling 125.3M — rather than on a subsample.

Compression improves at every step up the table, so a rule based on fertility alone would always select the largest candidate. At a 25M parameter budget that is the wrong conclusion. The embedding matrix has size vocab $\times\, d_{model}$ and is drawn from the same budget as the Transformer blocks: at 32,000 pieces it consumes 12.29M parameters, 49% of the budget, before a single attention head exists. Embedding parameters are also used inefficiently — a token reads one row and the remaining 31,999 do nothing for it — whereas every Transformer parameter participates in every token.

The selection rule applied, implemented in `tokenizer_stats.py`, is: reject any size whose embedding exceeds 34% of the parameter budget; reject any size using less than 95% of its vocabulary on held-out text; take the largest survivor. This rejects 24,000 and 32,000 on the budget constraint

Vocab	chars/token	fertility	byte fallback	UNK	vocabulary used	% of 25M budget
Gujarati (Model H)						
8,000	3.512	1.654	0.408%	0.0	98.1%	12%
12,000	3.723	1.560	0.432%	0.0	98.7%	18%
16,000	**3.865**	**1.503**	**0.449%**	0.0	**99.0%**	**25%**
24,000	4.046	1.436	0.470%	0.0	99.3%	37%
32,000	4.170	1.393	0.484%	0.0	99.4%	49%
Nepali (Model L)						
8,000	4.222	1.573	0.320%	0.0	98.2%	12%
12,000	4.508	1.473	0.342%	0.0	98.8%	18%
16,000	**4.689**	**1.416**	**0.356%**	0.0	**99.1%**	**25%**
24,000	4.925	1.348	0.373%	0.0	99.4%	37%
32,000	5.068	1.310	0.384%	0.0	99.5%	49%

Table 6: Vocabulary sweep, measured on the full validation split of each language. The two sweeps were run independently and reached the same answer.

in both languages and leaves **16,000**. Vocabulary utilisation rejects nothing in this sweep, since every candidate exceeds 98%.

Two further observations support the same conclusion. The marginal cost of compression rises sharply: moving from 8,000 to 12,000 buys 6.0% better compression at 256K parameters per percentage point, whereas moving from 24,000 to 32,000 buys 3.1% at 1.00M per point, roughly four times worse. And Phase 2 reports bits-per-byte, which is tokenizer-invariant by construction — a larger vocabulary improves perplexity while leaving bits-per-byte unchanged, so tuning the vocabulary against fertility optimises an artifact of the measurement rather than the model.

7.3 Unknown-token statistics

The UNK rate is 0.0 at every vocabulary size in both languages, and it carries no information. Because the tokenizers are configured with byte fallback, a character outside the vocabulary is emitted as `<0xNN>` byte pieces rather than as `<unk>`; any tokenizer configured this way scores zero by construction, so reporting it alone would be misleading.

The metric that measures coverage is the **byte-fallback rate**, given in Table 6: 0.449% for Gujarati and 0.356% for Nepali at the selected size. Both are well under 1%. What remains is symbols such as the degree sign, the rupee sign and `©` — characters no Indic vocabulary would be expected to contain, and which no vocabulary size would fix.

One measurement detail is worth recording. The tokenizer is fitted on a file holding one line per paragraph, so it never encounters a newline during training and has no piece for one. Encoding whole documents therefore charges every paragraph break to byte fallback: the same model on the same 41,171 documents reports 2.778% document-wise against **0.394% line-wise**, meaning roughly 87% of the document-wise figure is newline characters. Evaluation encodes line by line, matching the form the model was trained on.

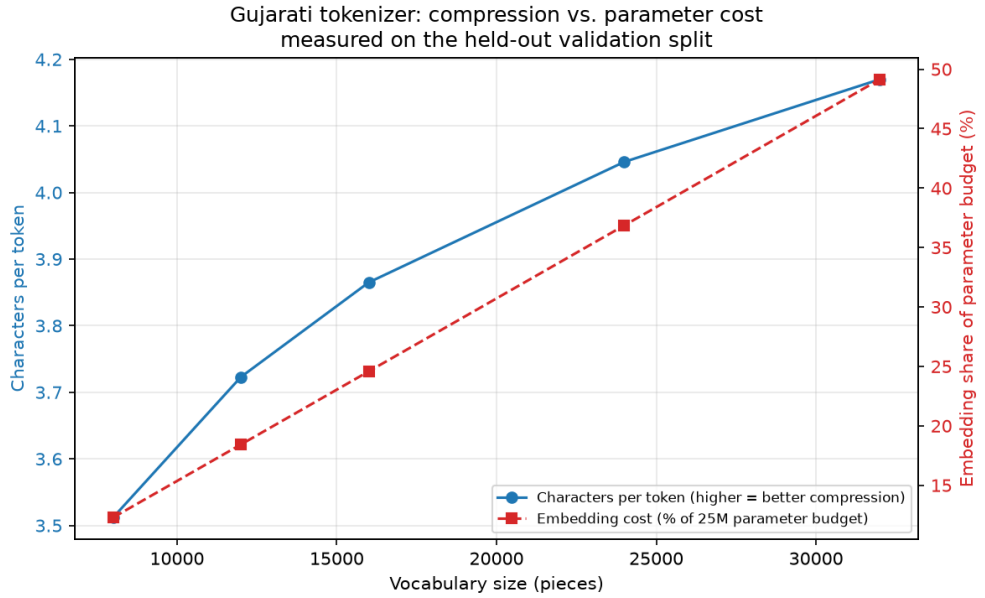


Figure 3: The Gujarati trade-off. Compression rises with vocabulary size, but embedding cost rises faster and is drawn from the same parameter budget as the Transformer blocks.

7.4 Tokenization examples

Three sentences per language, one per register. All are normalised in the same way as the corpus, so none can exhibit byte fallback on characters that preprocessing removes. _ marks a word boundary.

Gujarati

1. News prose with digits — 120 characters, 19 tokens, 6.32 chars/token, 1.19 tokens/word.

ગુજરાત સરકારે ૨૦૨૪ના શૈક્ષણિક વર્ષથી રાજ્યની તમામ પ્રાથમિક શાળાઓમાં માતૃભાષામાં શિક્ષણ ફરજિયાત બનાવવાનો નિર્ણય લીધો હતો.

_ગુજરાત | _સરકારે | _૨૦૨૪ | ના | _શૈક્ષણિક | _વર્ષથી | _રાજ્યની | _તમામ | _પ્રાથમિક | _શાળાઓમાં | _માતૃભાષા | માં | _શિક્ષણ | _ફરજિયાત | _બનાવવાનો | _નિર્ણય | _લીધો | _હતો | .

Nearly every word is a single token, and the year ૨૦૨૪ is one piece rather than four separate digits, which is the direct benefit of writing digits in the native script.

2. Academic prose — 120 characters, 24 tokens, 5.00 chars/token, 1.85 tokens/word.

આ સંશોધનપત્રમાં લેખકે ભાષાશાસ્ત્રીય દૃષ્ટિકોણથી ગુજરાતી ક્રિયાપદોના રૂપાંતરણની પ્રક્રિયાનું વિશ્લેષણ પ્રસ્તુત કર્યું છે.

_આ | _સંશોધન | પત્ર | માં | _લેખક | ે | _ભાષા | શાસ્ત્ર | ીય | _દૃષ્ટિકોણ | થી | _ગુજરાતી | _ક્રિયાપદ | ોના | _રૂપાંતર | ણ | ની | _પ્રક્રિયા | નું | _વિશ્લેષણ | _પ્રસ્તુત | _કર્યું | _છે | .

Compression falls because Sanskritic compounds are divided into their constituents: સંશોધનપત્રમાં becomes સંશોધન + પત્ર + માં, and ભાષાશાસ્ત્રીય becomes ભાષા + શાસ્ત્ર + ીય. These are the genuine morpheme boundaries, so the model receives reusable parts rather than one rare token.

3. Inflection paradigm — 119 characters, 34 tokens, 3.50 chars/token.

ઘર, ઘરમાં, ઘરોમાં, ઘરોમાંથી, ઘરવાળાં - એક જ મૂળ શબ્દમાંથી બનેલાં આ પાંચ રૂપો તપાસવાથી જણાય છે કે મૂળ

શબ્દ અકબંધ રહે છે.

_ઘર | , | _ઘરમાં | , | _ઘરોમાં | , | _ઘરોમાં | થી | , | _ઘર | વાળાં | _- | _એક | _જ | _મૂળ | _શબ્દ | માંથી | _બનેલા | ં | _આ | _પાંચ | _રૂપ | ો | _તપાસવા | થી | _જણાય | _છે | _કે | _મૂળ | _શબ્દ | _અકબંધ | _રહે | _છે | .

This is the argument for Unigram made visible. The stem ઘર is recoverable in all five forms, and ઘરોમાંથી is divided as ઘરોમાં + થી rather than at an arbitrary frequent boundary. Characters per token is low here only because the sentence is largely a list of short inflected forms.

Nepali

1. News prose with digits — 103 characters, 17 tokens, 6.06 chars/token, 1.06 tokens/word.

नेपाल सरकारले २०८१ सालको बजेटमा शिक्षा र स्वास्थ्य क्षेत्रका लागि छुट्याइएको रकम बढाउने निर्णय गरेको छ।

_नेपाल | _सरकारले | _२०८१ | _सालको | _बजेटमा | _शिक्षा | _र | _स्वास्थ्य | _क्षेत्रका | _लागि | _छुट्याइएको | _रकम | _बढाउने | _निर्णय | _गरेको | _छ | ।

Every word is a single token, including the long derived form छुट्याइएको. At 1.06 tokens per word this is close to the practical floor for a subword tokenizer.

2. Academic prose — 123 characters, 22 tokens, 5.59 chars/token, 1.69 tokens/word.

यस अनुसन्धानपत्रमा लेखकले भाषाशास्त्रीय दृष्टिकोणबाट नेपाली क्रियापदका रूपान्तरणको प्रक्रियाको विश्लेषण प्रस्तुत गरेका छन्।

_यस | _अनुसन्धान | पत्रमा | _लेखकले | _भाषा | शास्त्र | ीय | _दृष्टिकोण | बाट | _नेपाली | _क्रिया | पद | का | _रूपान्तरण | को | _प्रक्रिया | को | _विश्लेषण | _प्रस्तुत | _गरेका | _छन् | ।

The same compound-splitting behaviour appears, and on the same words: भाषा + शास्त्र + ीय matches the Gujarati segmentation exactly. The two tokenizers were trained separately on disjoint corpora, so this is convergence on shared Sanskritic morphology rather than any form of sharing between them.

3. Inflection paradigm — 118 characters, 34 tokens, 3.47 chars/token.

घर, घरमा, घरहरूमा, घरहरूबाट, घरवालाहरू - एउटै मूल शब्दबाट बनेका यी पाँच रूपहरू हेर्दा मूल शब्द अक्षुण्ण रहेको देखिन्छ।

_घर | , | _घरमा | , | _घर | हरूमा | , | _घर | हरूबाट | , | _घर | वाला | हरू | _- | _एउटै | _मूल | _शब्द | बाट | _बनेका | _यी | _पाँच | _रूप | हरू | _हेर्दा | _मूल | _शब्द | _अ | क्ष | ु | ण् | ण | _रहेको | _देखिन्छ | ।

Nepali divides this paradigm even more cleanly than Gujarati: घर is an independent token in four of the five forms, with the plural marker हरू and the case endings मा and बाट carried as separate pieces. The rare word अक्षुण्ण is divided into five pieces, which illustrates the expected behaviour of a 16,000-piece vocabulary on genuinely uncommon words.

7.5 Training sample

The tokenizer is fitted on a proportional sample of the training split rather than on all of it: 210.5M characters for Gujarati (10.8% of the split) and 248.4M for Nepali (10.9%). The constraint is memory. SentencePiece constructs a suffix array whose peak footprint is a large multiple of the input size, and the dominant parameter proved to be `seed_sentencepiece_size`, which defaults to one million candidate pieces; reducing it to 200,000 cut peak resident memory by roughly a factor of ten with no measurable effect on the resulting vocabulary. Beyond approximately 10% the machine begins paging: a 15% sample required 31.5 minutes per model against 6.2 minutes at 10%, a fivefold increase for a sample only 50% larger.

The sample is drawn proportionally across sources. This is necessary rather than cosmetic —

reading shards in filename order would have fitted the tokenizer to essentially a single source. **This limit applies only to fitting the vocabulary**: all token counts, all manual shares and every metric reported here are computed over 100% of the relevant split.

8 Deliverables

Required	**Location**
Dataset collection scripts	`<lang>/data/collection/`
Dataset preprocessing pipelines	`<lang>/data/preprocessing/`
Per-language dataset statistics	`report/phase1/<lang>/README.md`
Train/validation/test splits	`<lang>/data/splits/`, Drive link in README
Tokenizer training code	`<lang>/tokenizer/train_tokenizer.py`
Vocabulary files	`<lang>/tokenizer/vocab/16k_unigram_*.vocab`
Tokenizer model files	`<lang>/tokenizer/model/16k_unigram_*.model`

Tokenizer models and vocabularies are small enough to be committed to the repository directly. The corpora are provided through Google Drive as plain readable `.jsonl` files, with the link above and in the repository README, as the submission policy requires for large artifacts. Cross-language tables and figures are under `report/phase1/tables/` and `report/phase1/figures/`, and the machine-written JSON summaries from which every figure in this report is derived are under `report/phase1/<lang>/json/`.